



TAREA #3

PRUEBAS Y REFACTORING

TEMA:

SISTEMA SPORTSPREDICTOR

GRUPO:

5

INTEGRANTES:

NAVARRETE FIGUEROA JOSÉ ANTONIO

ZOUEIN VÉLEZ NEJEH YOUSSEF

INGA ONTANEDA ANGIE LILIANA

BORBOR CRESPIAN KEVIN ANDRES

ASIGNATURA:

DISEÑO DE SOFTWARE

PARALELO:

P1

PROFESOR:

Mgtr. JURADO MOSQUERA DAVID ALONSO

Tabla de contenido

1. Plan de Pruebas.....	3
2. Detección de Code Smells.....	6
Enlace del repositorio.....	8

1. Plan de Pruebas

Clase	ID	Método a probar	Datos de entrada	Salida esperada	Propósito
creadorPronosticoFutbol	CP-01	crearPronostico(Evento , Usuario, Object datos)	EventoFutbol abierto, datos = "EquipoA"	Crea pronosticoFutbol ; estado inicial "PENDIENTE"	Verificar que se pueda crear correctamente un propósito de fútbol cuando el pronóstico está abierto y los datos son válidos.
creadorPronosticoFutbol	CP-02	crearPronostico(Evento , Usuario, Object datos)	EventoFutbol cerrado, datos = "EquipoA"	Lanza IllegalStateException	Validar que no se pueda crear pronóstico si el evento no está abierto.
creadorPronosticoFutbol	CP-03	crearPronostico(Evento , Usuario, Object datos)	evento no es EventoFutbol	Lanza IllegalArgumentException	Verificar conversión de tipos en el factory method.
pronosticoFutbol	CP-04	evaluar(String resultado)	"3-1" y predicción EquipoLocal	ACERTADO; calcularPuntos() == 10	Validar evaluación de resultado de fútbol correcto.
pronosticoFutbol	CP-05	evaluar(String resultado)	"3x1" (formato inválido)	EN_REVISION; calcularPuntos() == 0	Asegurar manejo de formato inválido como error esperado.
pronosticoFutbol	CP-06	evaluar(String resultado)	"1-2" y predicción EquipoLocal	FALLIDO; calcularPuntos() == 0	Verificar resultado incorrecto sin otorgar puntos.

pronosticoBalon cesto	CP-07	evaluar(String resultado)	"EquipoA" = predicción	ACERTADO; calcularPuntos() == 10; usuario +10	Verificar evaluación correcta de pronóstico de baloncesto.
pronosticoBalon cesto	CP-08	evaluar(String resultado)	resultado =null o resultado diferente	FALLIDO; calcularPuntos() == 0	Verificar el comportamiento ante un resultado nulo.
pronosticoTenis	CP-09	evaluar(String resultado)	"Jugador 1,3"; predicción "Jugador 1,3"	ACERTADO; calcularPuntos() == 10; usuario +10	Validar puntos adicionales por sets correctos.
pronosticoTenis	CP-10	evaluar(String resultado)	"Jugador 2,3" con predicción "Jugador 1,3"	FALLIDO; calcularPuntos() == 0	Verificar fallo cuando el ganador real no coincide.
pronosticoTenis	CP-11	evaluar(String resultado)	"Jugador 1" (sin coma)	EN_REVISION; calcularPuntos() == 0	Caso límite de entrada incompleta.
Usuario	CP-12	agregarPuntos(int puntos)	puntos = 15 sobre usuario con 0	getPuntos() == 15	Verificar acumulación de puntos.
Usuario	CP-13	toString()	usuario con nombre y 5 puntos	Devuelve texto con nombre y puntos	Confirmar formato de cadena.
Evento	CP-14	cerrarPronosticos() + estaAbierto()	Evento abierto	estaAbierto() == false; getEstado() == CERRADO	Verificar cierre de evento.

Evento	CP-15	registrarResultado(String resultado)	Evento abierto	getEstado() == FINALIZADO	Verificar cambio de estado al registrar resultado.
Administrador	CP-16	publicarResultado(Evento, String resultado)	Evento abierto	Evento pasa a FINALIZADO	Verificar que admin puede publicar resultados.
GestorNotificaciones	CP-17	suscribir(ObservadorNotificacion observador) + notificar(Notificacion)	Observer suscrito	Llamada a actualizar(Notificacion notificacion) de todos los observadores	Verificar notificación a observadores.
GestorNotificaciones	CP-18	desuscribir(ObservadorNotificacion observador) + notificar(Notificacion)	Observer removido	Observer no recibe notificación	Confirmar que desuscribir funciona.
ProveedorEstadisticasAdapter	CP-19	obtenerEstadisticas(String eventold)	eventold = "evt-001"	Lista con 1 estadística y id = evt-001	Verificar adaptación del proveedor externo.
ProveedorEstadisticasAdapter	CP-20	obtenerTendencias(String eventold)	eventold = "evt-001"	Lista con 1 tendencia	Validar método de tendencias.
SistemaSportsPredictor	CP-21	consultarEstadisticas(String eventold)	eventold = "evt-001"	Retorna lista desde servicio	Verificar delegación de consulta de estadísticas.
SistemaSportsPredictor	CP-22	realizarPronostico(Evento, Usuario, datos)	Evento abierto de fútbol	Devuelve pronóstico PENDIENTE	Confirmar creación de pronóstico vía sistema.
SistemaSportsPredictor	CP-23	publicarResultado(Pronostico, String resultado)	Pronóstico pendiente	Pronóstico evaluado y notificación enviada	Validar flujo completo de publicación de resultados.

ManejadorControlCalidad	CP-24	manejar(ReporteIncidencia reporte)	Reporte registrado	Mensajes de revisión y ajuste	Verificar que el handler procesa el reporte.
ReporteIncidencia	CP-25	actualizarEstado(EstadoReporte)	Estado EN_REVISION	getEstado() == EN_REVISION	Verificar actualización de estado de reporte.
ReporteIncidencia	CP-26	cerrar()	Reporte registrado	getEstado() == RESUELTO	Verificar que un reporte puede cerrarse.
ReglaPuntuacion	CP-27	validar()	puntosPorAcierto = 5	true	Verificar regla válida.
ReglaPuntuacion	CP-28	validar()	puntosPorAcierto = 0	false	Verificar regla inválida.
UsuarioSistema	CP-29	iniciarSesion()	Usuario cualquiera	true	Confirmar acceso al sistema (simulado).
UsuarioSistema	CP-30	cerrarSesion()	Usuario cualquiera	No lanza excepción	Validar cierre de sesión sin error.

2. Detección de Code Smells

1.Data clumps

Descripción: En las clases EventoFutbol y EventoBaloncesto se pueden observar un grupo de variables que se repiten, equipoLocal y equipoVisitante, las cuales siempre se observan en los mismos parametros de los metodos, lo cual podria significar que en realidad son un grupo de datos y se debería combinarlo en un objeto. También se ve el mismo smell en la clase EventoTenis con las variables jugadorDos y jugadorUno.

Impacto: El impacto de este code smell en el código, vuelve los métodos más complejos, ya que aumenta la cantidad de datos en los parámetros, haciéndolos difíciles de manejar, incluso de entenderlos.

Ubicación:

- EventoFutbol.java Lineas 11-16 en el parametro del constructor
- EventoBaloncesto.java Lineas 11-16 en el parametro del constructor
- EventoTenis.java Lineas 11-16 en el parametro del constructor

Evidencia:

```

public class EventoFutbol extends Evento {
    private String equipoLocal;
    private String equipoVisitante;
    private int golesLocal;
    private int golesVisitante;

    public EventoFutbol(String id, String nombre, LocalDateTime fechaInicio,
                        String equipoLocal, String equipoVisitante) {
        super(id, nombre, fechaInicio);
        this.equipoLocal = equipoLocal;
        this.equipoVisitante = equipoVisitante;
    }
}

public class EventoBaloncesto extends Evento {
    private String equipoLocal;
    private String equipoVisitante;
    private int puntosLocal;
    private int puntosVisitante;

    public EventoBaloncesto(String id, String nombre, LocalDateTime fechaInicio,
                           String equipoLocal, String equipoVisitante) {
        super(id, nombre, fechaInicio);
        this.equipoLocal = equipoLocal;
        this.equipoVisitante = equipoVisitante;
    }
}

public class EventoTenis extends Evento {
    private String jugadorUno;
    private String jugadorDos;
    private int setsJugadorUno;
    private int setsJugadorDos;

    public EventoTenis(String id, String nombre, LocalDateTime fechaInicio,
                      String jugadorUno, String jugadorDos) {
        super(id, nombre, fechaInicio);
        this.jugadorUno = jugadorUno;
        this.jugadorDos = jugadorDos;
    }
}

```

Técnica de refactorización: Se debería utilizar extract class para mover las variables hacia el nuevo objeto que se creara para contener este grupo de datos.

2.Long Parameter List

Descripción: En las clases EventoFutbol, EventoBaloncesto y EventoTenis, se puede observar que en los constructores hay más de 4 parámetros, lo cual es un indicio de un smell como Long Parameter List.

Impacto: Si se sigue manteniendo la misma cantidad de parámetros en los constructores, esto podría resultar en problemas a futuro, no solo por aumentar la complejidad para leer el código, además podría ocasionar errores al llamarlo en un main y colocar parámetros equivocados de lugar.

Ubicación:

- EventoFutbol.java Lineas 11-16 en el parametro del constructor
- EventoBaloncesto.java Lineas 11-16 en el parametro del constructor
- EventoTenis.java Lineas 11-16 en el parametro del constructor

Evidencia:

```
public class EventoFutbol extends Evento {
    private String equipoLocal;
    private String equipoVisitante;
    private int golesLocal;
    private int golesVisitante;

    public EventoFutbol(String id, String nombre, LocalDateTime fechaInicio,
                        String equipoLocal, String equipoVisitante) {
        super(id, nombre, fechaInicio);
        this.equipoLocal = equipoLocal;
        this.equipoVisitante = equipoVisitante;
    }
}
```

```
public class EventoFutbol extends Evento {
    private String equipoLocal;
    private String equipoVisitante;
    private int golesLocal;
    private int golesVisitante;

    public EventoFutbol(String id, String nombre, LocalDateTime fechaInicio,
                        String equipoLocal, String equipoVisitante) {
        super(id, nombre, fechaInicio);
        this.equipoLocal = equipoLocal;
        this.equipoVisitante = equipoVisitante;
    }
}
```

```
public class EventoTenis extends Evento {
    private String jugadorUno;
    private String jugadorDos;
    private int setsJugadorUno;
    private int setsJugadorDos;

    public EventoTenis(String id, String nombre, LocalDateTime fechaInicio,
                      String jugadorUno, String jugadorDos) {
        super(id, nombre, fechaInicio);
        this.jugadorUno = jugadorUno;
        this.jugadorDos = jugadorDos;
    }
}
```

Técnica de refactorización: La técnica que se podría utilizar es Introduce Parameter Object ya que se puede reducir el número de parámetros, convirtiendo

cierto grupo de datos en un objeto, especial si es que se repiten como los strings dentro del parámetro del constructor.

3. Shotgun Surgery

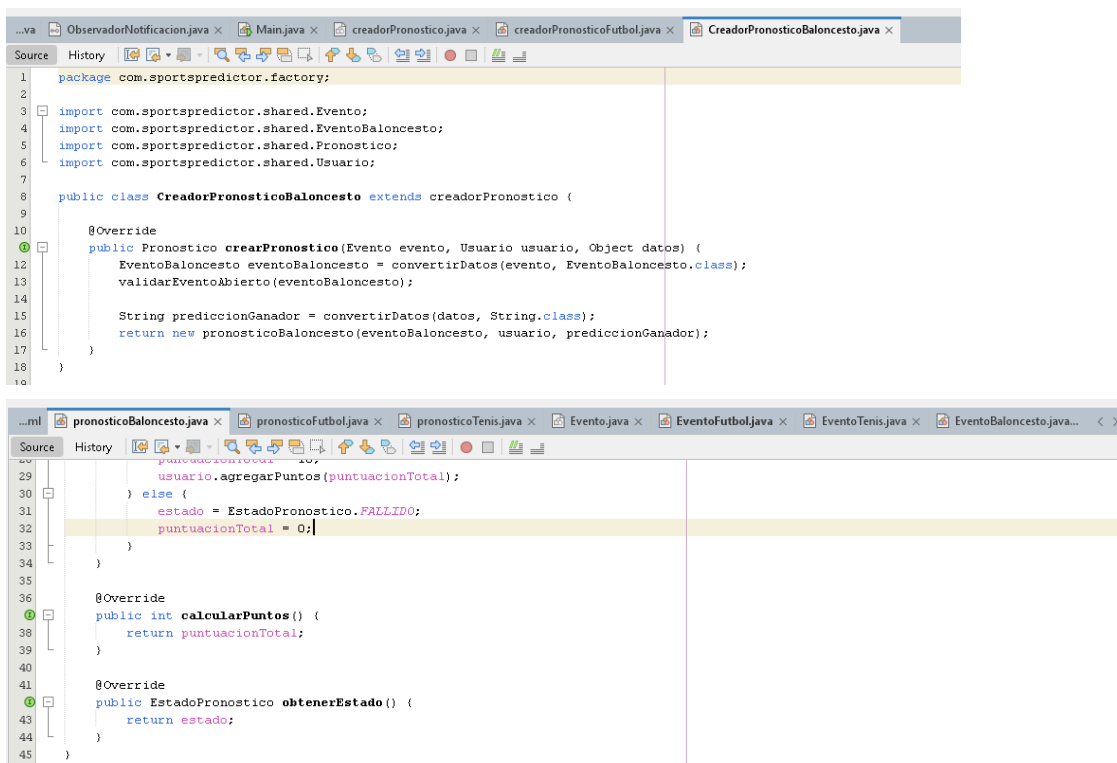
Descripción: Si realizamos algún tipo modificación, como agregar un nuevo deporte, entonces estaríamos afectando y realizando cambios en diferentes clases, en este caso son cuatro clases diferentes, Evento, pronostico, creadorPronostico y en el main.

Impacto: Si se quiere agregar un nuevo deporte como ping pong, se estarían realizando diferentes cambios, se necesitaría crear EventoPingPong, pronosticoPingPong, creadorPronosticoPingPong y además modificar el main si es que se quisiera registrar un nuevo pronóstico para ping pong. Esto podría causar algún problema si es que no se crea alguna de estas clases.

Ubicación:

- EventoBaloncesto.java, pronosticoBaloncesto.java, creadorPronosticoBaloncesto.java
- EventoFutbol.java, pronosticoFutbol.java, creadorPronosticoFutbol.java
- EventoTenis.java, pronosticoTenis.java, creadorPronosticoTenis.java

Evidencia:



```
package com.sportspredictor.factory;

import com.sportspredictor.shared.Evento;
import com.sportspredictor.shared.EventoBaloncesto;
import com.sportspredictor.shared.Pronostico;
import com.sportspredictor.shared.Usuario;

public class CreadorPronosticoBaloncesto extends creadorPronostico {

    @Override
    public Pronostico crearPronostico(Evento evento, Usuario usuario, Object datos) {
        EventoBaloncesto eventoBaloncesto = convertirDatos(evento, EventoBaloncesto.class);
        validarEventoAbierto(eventoBaloncesto);

        String prediccionGanador = convertirDatos(datos, String.class);
        return new pronosticoBaloncesto(eventoBaloncesto, usuario, prediccionGanador);
    }
}

// ...

usuario.agregarPuntos(puntuacionTotal);
} else {
    estado = EstadoPronostico.FALLIDO;
    puntuacionTotal = 0;
}

@Override
public int calcularPuntos() {
    return puntuacionTotal;
}

@Override
public EstadoPronostico obtenerEstado() {
    return estado;
}
```

```
--va pronosticofutbol.java x pronosticotenis.java x Evento.java x Eventofutbol.java x Eventotenis.java x EventoBaloncesto.java x UsuarioSistema.java...
Source History
1 package com.sportspredictor.shared;
2
3 import java.time.LocalDateTime;
4
5 public class EventoBaloncesto extends Evento {
6     private String equipoLocal;
7     private String equipoVisitante;
8     private int puntosLocal;
9     private int puntosVisitante;
10
11     public EventoBaloncesto(String id, String nombre, LocalDateTime fechaInicio,
12                             String equipoLocal, String equipoVisitante) {
13         super(id, nombre, fechaInicio);
14         this.equipoLocal = equipoLocal;
15         this.equipoVisitante = equipoVisitante;
16     }
17
18     public String getEquipoLocal() { return equipoLocal; }
19     public String getEquipoVisitante() { return equipoVisitante; }
20
21 }
```

```
--va ManejadorIncidente.java x ObservadorNotificacion.java x Main.java x creadorPronostico.java x creadorPronosticoFutbol.java x CreadorPronosticoBaloncesto.java...
Source History
1 package com.sportspredictor.factory;
2
3 import com.sportspredictor.shared.Evento;
4 import com.sportspredictor.shared.EventoFutbol;
5 import com.sportspredictor.shared.Pronostico;
6 import com.sportspredictor.shared.Usuario;
7
8 public class creadorPronosticoFutbol extends creadorPronostico {
9     @Override
10     public Pronostico crearPronostico(Evento evento, Usuario usuario, Object datos) {
11         EventoFutbol eventoFutbol = convertirDatos(evento, EventoFutbol.class);
12         validarEventoAbierto(eventoFutbol);
13
14         String prediccionGanador = convertirDatos(datos, String.class);
15         return new pronosticoFutbol(eventoFutbol, usuario, prediccionGanador);
16     }
17
18 }
```

```
--va pronosticofutbol.java x pronosticotenis.java x Evento.java x Eventofutbol.java x Eventotenis.java x EventoBaloncesto.java x UsuarioSistema.java...
Source History
46 } else {
47     ganadorReal = "EMPATE";
48 }
49
50 if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
51     estado = EstadoPronostico.ACERTADO;
52     usuario.agregarPuntos(10);
53 } else {
54     estado = EstadoPronostico.FALLIDO;
55 }
56
57
58 @Override
59 public int calcularPuntos() {
60     return estado == EstadoPronostico.ACERTADO ? 10 : 0;
61 }
62
63 @Override
64 public EstadoPronostico obtenerEstado() {
65     return estado;
66 }
67
68 }
```

```
--va pronosticofutbol.java x pronosticotenis.java x Evento.java x Eventofutbol.java x Eventotenis.java x EventoBaloncesto.java x UsuarioSistema.java...
Source History
1 package com.sportspredictor.shared;
2
3 import java.time.LocalDateTime;
4
5 public class Eventofutbol extends Evento {
6     private String equipoLocal;
7     private String equipoVisitante;
8     private int golesLocal;
9     private int golesVisitante;
10
11     public Eventofutbol(String id, String nombre, LocalDateTime fechaInicio,
12                        String equipoLocal, String equipoVisitante) {
13         super(id, nombre, fechaInicio);
14         this.equipoLocal = equipoLocal;
15         this.equipoVisitante = equipoVisitante;
16     }
17
18     public String getEquipoLocal() { return equipoLocal; }
19     public String getEquipoVisitante() { return equipoVisitante; }
20
21 }
```

```
...va Main.java x creadorPronostico.java x creadorPronosticoFutbol.java x CreadorPronosticoBaloncesto.java x creadorPronosticoTenis.java x
Source History
1 package com.sportspredictor.factory;
2
3 import com.sportspredictor.shared.Evento;
4 import com.sportspredictor.shared.EventoTenis;
5 import com.sportspredictor.shared.Pronostico;
6 import com.sportspredictor.shared.Usuario;
7
8 public class creadorPronosticoTenis extends creadorPronostico {
9     @Override
10     public Pronostico crearPronostico(Evento evento, Usuario usuario, Object datos) {
11         EventoTenis eventoTenis = convertirDatos(evento, EventoTenis.class);
12         validarEventoAbierto(eventoTenis);
13
14         String prediccionGanador = convertirDatos(datos, String.class);
15         return new pronosticoTenis(eventoTenis, usuario, prediccionGanador);
16     }
17 }
18

...va pronosticoTenis.java x Evento.java x EventoFutbol.java x EventoTenis.java x EventoBaloncesto.java x UsuarioSistema.java x Usuario.java x Manejad...
Source History
40
41 @Override
42 public void evaluar(String resultado) {
43     if (resultado == null || !resultado.contains(",")) {
44         estado = EstadoPronostico.EN_REVISION;
45         return;
46     }
47
48     String[] partes = resultado.trim().split(",");
49     String ganadorReal = partes[0].trim();
50     setsResultado = partes.length > 1 ? parseSets(partes[1].trim()) : 0;
51
52     if (ganadorReal.equalsIgnoreCase(ganadorPronosticado)) {
53         estado = EstadoPronostico.ACERTADO;
54         usuario.agregarPuntos(5 + (setsResultado == setsPronosticados ? 5 : 0));
55     } else {
56         estado = EstadoPronostico.FALLIDO;
57     }
58 }
59
60 @Override
61 public int calcularPuntos() {
62     if (estado == EstadoPronostico.ACERTADO) {
63         return 5 + (setsResultado == setsPronosticados ? 5 : 0);
64     }
65     return 0;
66 }

...va pronosticoTenis.java x Evento.java x EventoFutbol.java x EventoTenis.java x EventoBaloncesto.java x UsuarioSistema.java x Usuario.java x Manejad...
Source History
1 package com.sportspredictor.shared;
2
3 import java.time.LocalDateTime;
4
5 public class EventoTenis extends Evento {
6     private String jugadorUno;
7     private String jugadorDos;
8     private int setsJugadorUno;
9     private int setsJugadorDos;
10
11     public EventoTenis(String id, String nombre, LocalDateTime fechaInicio,
12         String jugadorUno, String jugadorDos) {
13         super(id, nombre, fechaInicio);
14         this.jugadorUno = jugadorUno;
15         this.jugadorDos = jugadorDos;
16     }
17
18     public String getJugadorUno() { return jugadorUno; }
19     public String getJugadorDos() { return jugadorDos; }
20 }
21
```

Técnica de refactorización: La técnica de refactorización a utilizar debería ser una combinación de move method y move field, para que se muevan los comportamientos de esas clases ya existentes a una sola clase.

4. Primitive Obsession

Descripción: Se utilizan datos primitivos como Strings en la clase EventoFutbol y EventoTenis, del dato resultado, en el método evaluar se separa en partes el string resultado, cuando estos podrían ser tratados como objetos con su propio comportamiento y sin necesidad de ser separados.

Impacto: En este código se podrían formar varios problemas al utilizar datos primitivos como el string para resultado, ya que si en algún caso se llega a colocar un formato diferente, esto causaría que el programa se caiga o que se obtenga un estado de pronóstico distinto al verdadero.

Ubicación:

- pronosticoFutbol.java líneas 32-39 dentro del metodo evaluar
- pronosticoTenis líneas 33-39 dentro del método parseSets
- pronosticoTenis líneas 48-50 dentro del método evaluar

Evidencia:

```
25  @Override
26  public void evaluar(String resultado) {
27      if (resultado == null || !resultado.contains("-")) {
28          estado = EstadoPronostico.EN_REVISION;
29          return;
30      }
31
32      String[] partes = resultado.trim().split("-");
33      try {
34          marcadorLocal = Integer.parseInt(partes[0].trim());
35          marcadorVisitante = Integer.parseInt(partes[1].trim());
36      } catch (NumberFormatException e) {
37          estado = EstadoPronostico.EN_REVISION;
38          return;
39      }
40
41      String ganadorReal;
42      if (marcadorLocal > marcadorVisitante) {
43          ganadorReal = evento.getEquipoLocal();
44      } else if (marcadorLocal < marcadorVisitante) {
45          ganadorReal = evento.getEquipoVisitante();
46      } else {
47          ganadorReal = "EMPATE";
48      }
49
50      if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
51          estado = EstadoPronostico.ACERTADO;
52          return;
53      }
54      estado = EstadoPronostico.EN_REVISION;
55  }
```

```
32
33  private int parseSets(String texto) {
34      try {
35          return Integer.parseInt(texto);
36      } catch (NumberFormatException e) {
37          return 0;
38      }
39  }
```

```
43  public void evaluar(String resultado) {
44      if (resultado == null || !resultado.contains(",")) {
45          estado = EstadoPronostico.EN_REVISION;
46          return;
47      }
48
49      String[] partes = resultado.trim().split(",");
50      String ganadorReal = partes[0].trim();
51      setsResultado = partes.length > 1 ? parseSets(partes[1].trim()) : 0;
52  }
```

Técnica de refactorización: La técnica que se utilizaría para este smell es Replace Primitive with Object, se crearán las clases ResultadoFutbol y ResultadoTenis, así evitando que se usen datos primitivos como strings y se utilice los nuevos objetos.

5.Feature Envy

Descripción: En la clase pronosticoFutbol se está accediendo a los datos de otras clases más que sus propios datos, ya que se está utilizando datos de Evento y de Usuario en métodos de esta clase.

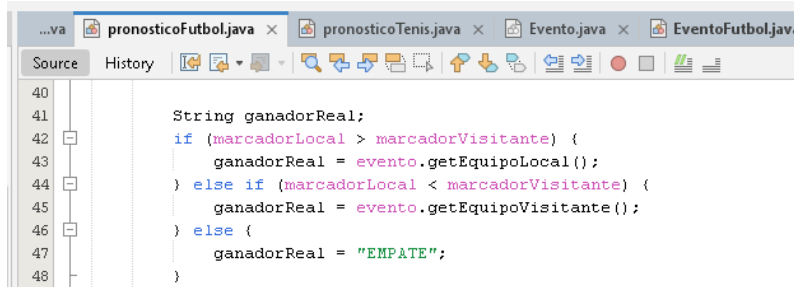
Impacto: Debido a que el método evaluar() de pronosticoFutbol utiliza muchos datos que no son de su propia clase, se están mezclando responsabilidades de otras

clases, como por ejemplo se evalúa la predicción y además se calcula el resultado del evento.

Ubicación:

- pronosticoFutbol.java líneas 41-48 dentro del metodo evaluar

Evidencia:



```
40
41
42     String ganadorReal;
43     if (marcadorLocal > marcadorVisitante) {
44         ganadorReal = evento.getEquipoLocal();
45     } else if (marcadorLocal < marcadorVisitante) {
46         ganadorReal = evento.getEquipoVisitante();
47     } else {
48         ganadorReal = "EMPATE";
49     }
```

Técnica de refactorización: La técnica para refactorizar debería ser Extract Method, ya que solo se necesita extraer un fragmento del código dentro del método.

6. Dead code

Descripción: El campo url se recibe y se guarda en el constructor de ProveedorDatosExterno, pero nunca se usa en solicitarDatos() ni en ningún otro método, por lo tanto es una variable que no se está utilizando en la clase.

Impacto: Da la falsa impresión de que la URL influye en el resultado, cuando en realidad no hace nada. Es peso innecesario que puede confundir al mantener el código.

Ubicación:

- ProveedorDatosExterno.java — campo private String url; y parámetro url del constructor.

```

package com.sportspredictor.adapter;

/**
 * Simula una API externa de estadísticas deportivas (Adaptee).
 * En un caso real, aquí iría la llamada HTTP al proveedor.
 */
public class ProveedorDatosExterno {
    private String url;

    public ProveedorDatosExterno(String url) {
        this.url = url;
    }

    public DatosProveedor solicitarDatos(String eventoId) {
        String contenidoSimulado = "{\"posesion\":55,\"tirosAlArco\":8}";
        return new DatosProveedor(eventoId, contenidoSimulado);
    }
}

```

Técnica de refactorización: Eliminar el campo y el parámetro si no se van a usar.

7. Alternative Classes with Different Interfaces

Descripción: pronosticoBaloncesto guarda puntuacionTotal como campo y lo retorna en calcularPuntos(), mientras pronosticoFutbol recalcula desde estado.

Impacto: dificulta tratar las 3 implementaciones de forma uniforme/polimórfica.

Ubicación: factory/pronosticoFutbol.java, pronosticoBaloncesto.java, pronosticoTenis.java, CreadorPronosticoBaloncesto.java.

```

src > com > sportspredictor > factory > pronosticoBaloncesto.java
8   public class pronosticoBaloncesto extends AbstractPronostico {
13   public pronosticoBaloncesto(EventoBaloncesto evento, Usuario usuario) {
16       this.prediccionGanador = prediccionGanador;
17       this.puntuacionTotal = 0;
18   }
19
20   @Override
21   public void evaluar(String resultado) {
22       String ganadorReal = resultado != null ? resultado.trim() : "";
23       if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
24           puntuacionTotal = 10;
25           registrarAcierto(puntuacionTotal);
26       } else {
27           puntuacionTotal = 0;
28           registrarFallo();
29       }
30   }
31
32   @Override
33   public int calcularPuntos() {
34       return puntuacionTotal;
35   }
36 }

```

Técnica de refactorización: Unificar Interfaces (Unify Interfaces)

8.

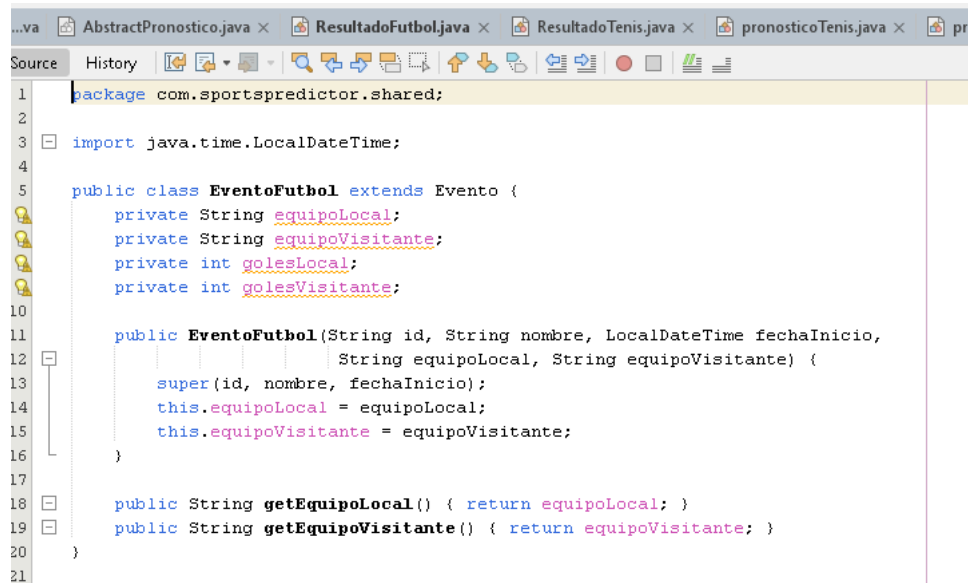
3. Técnicas de Refactorización

Refactor Extract Class

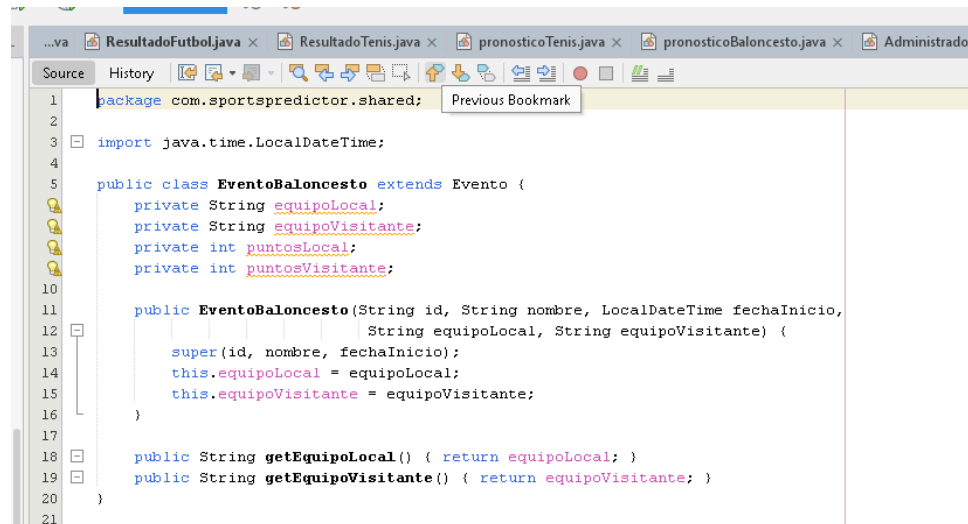
1. Code Smell corregido: Data Clumps

2. Fragmento de código antes y después de la refactorización

Antes:



```
1 package com.sportspredictor.shared;
2
3 import java.time.LocalDateTime;
4
5 public class EventoFutbol extends Evento {
6     private String equipoLocal;
7     private String equipoVisitante;
8     private int golesLocal;
9     private int golesVisitante;
10
11     public EventoFutbol(String id, String nombre, LocalDateTime fechaInicio,
12                        String equipoLocal, String equipoVisitante) {
13         super(id, nombre, fechaInicio);
14         this.equipoLocal = equipoLocal;
15         this.equipoVisitante = equipoVisitante;
16     }
17
18     public String getEquipoLocal() { return equipoLocal; }
19     public String getEquipoVisitante() { return equipoVisitante; }
20 }
21
```



```
1 package com.sportspredictor.shared;
2
3 import java.time.LocalDateTime;
4
5 public class EventoBaloncesto extends Evento {
6     private String equipoLocal;
7     private String equipoVisitante;
8     private int puntosLocal;
9     private int puntosVisitante;
10
11     public EventoBaloncesto(String id, String nombre, LocalDateTime fechaInicio,
12                           String equipoLocal, String equipoVisitante) {
13         super(id, nombre, fechaInicio);
14         this.equipoLocal = equipoLocal;
15         this.equipoVisitante = equipoVisitante;
16     }
17
18     public String getEquipoLocal() { return equipoLocal; }
19     public String getEquipoVisitante() { return equipoVisitante; }
20 }
21
```

```

Source History
1 package com.sportspredictor.shared;
2
3 import java.time.LocalDateTime;
4
5 public class EventoTenis extends Evento {
6     private String jugadorUno;
7     private String jugadorDos;
8     private int setsJugadorUno;
9     private int setsJugadorDos;
10
11     public EventoTenis(String id, String nombre, LocalDateTime fechaInicio,
12         String jugadorUno, String jugadorDos) {
13         super(id, nombre, fechaInicio);
14         this.jugadorUno = jugadorUno;
15         this.jugadorDos = jugadorDos;
16     }
17
18     public String getJugadorUno() { return jugadorUno; }
19     public String getJugadorDos() { return jugadorDos; }
20 }

```

Después:

```

10 L */
11 public class Equipos {
12     private final String nombreLocal;
13     private final String nombreVisitante;
14
15     public Equipos(String nombreLocal, String nombreVisitante){
16         this.nombreLocal = nombreLocal;
17         this.nombreVisitante = nombreVisitante;
18     }
19     public String getNombreLocal(){return nombreLocal;}
20     public String getNombreVisitante(){return nombreVisitante;}
21
22     @Override
23     public String toString(){return nombreLocal + " vs " + nombreVisitante;}
24
25 }

```

```

2
3 import java.time.LocalDateTime;
4
5 public class EventoBaloncesto extends Evento {
6     private final Equipos equipos;
7     private int puntosLocal;
8     private int puntosVisitante;
9
10     public EventoBaloncesto(String id, String nombre, LocalDateTime fechaInicio,
11         Equipos equipos) {
12         super(id, nombre, fechaInicio);
13         this.equipes = equipos;
14     }
15
16     public Equipos getEquipos() { return equipos; }
17     public String getEquipoLocal() { return equipos.getNombreLocal(); }
18     public String getEquipoVisitante() { return equipos.getNombreVisitante(); }
19 }

```

```

5 public class EventoFutbol extends Evento {
6     private final Equipos equipos;
7     private int golesLocal;
8     private int golesVisitante;
9
10     public EventoFutbol(String id, String nombre, LocalDateTime fechaInicio,
11         Equipos equipos) {
12         super(id, nombre, fechaInicio);
13         this.equipes = equipos;
14     }
15
16     public Equipos getEquipos() { return equipos; }
17     public String getEquipoLocal() { return equipos.getNombreLocal(); }
18     public String getEquipoVisitante() { return equipos.getNombreVisitante(); }
19 }

```

```

5 public class EventoTenis extends Evento {
6     private final Equipos equipos;
7     private int setsJugadorUno;
8     private int setsJugadorDos;
9
10    public EventoTenis(String id, String nombre, LocalDateTime fechaInicio,
11                       Equipos equipos) {
12        super(id, nombre, fechaInicio);
13        this.equipos = equipos;
14    }
15
16    public Equipos getEquipos() { return equipos; }
17    public String getJugadorUno() { return equipos.getNombreLocal(); }
18    public String getJugadorDos() { return equipos.getNombreVisitante(); }
19 }

```

3. Justificación del cambio: El cambio se realizó ya que existían grupos de datos que siempre se encontraban juntos en diferentes parametros de estas clases, EventoFutbol, EventoBaloncesto y EventoTenis, se aplicó Extract Class para resolver este smell de data clumps, convirtiendo el grupo de datos en una clase concreta llamada Equipos, gracias a esto se mejoró la legibilidad del código, además de eliminar la duplicación de código entre las clases.

4. Verificación mediante pruebas: Se realizó la verificación mediante las pruebas CP-09, CP-10, CP-11, CP-14 y CP-15, ya que en todas estas pruebas se construye el evento con new Equipos("EquipoLocal", "EquipoVisitante"), al realizar las pruebas se verifica que este cambio no afecta al resultado original y sigue retornando los mismos resultados, por lo tanto esta refactorización esta comprobada.

Introduce Parameter Object

1. Code smell corregido: Long Parameter List

2. Fragmento de código antes y después de la refactorización

Antes:

```

11 public EventoBaloncesto(String id, String nombre, LocalDateTime fechaInicio,
12                        String equipoLocal, String equipoVisitante) {
13     super(id, nombre, fechaInicio);
14     this.equipoLocal = equipoLocal;
15     this.equipoVisitante = equipoVisitante;
16 }

11 public EventoFutbol(String id, String nombre, LocalDateTime fechaInicio,
12                     String equipoLocal, String equipoVisitante) {
13     super(id, nombre, fechaInicio);
14     this.equipoLocal = equipoLocal;
15     this.equipoVisitante = equipoVisitante;
16 }

11 public EventoTenis(String id, String nombre, LocalDateTime fechaInicio,
12                    String jugadorUno, String jugadorDos) {
13     super(id, nombre, fechaInicio);
14     this.jugadorUno = jugadorUno;
15     this.jugadorDos = jugadorDos;
16 }

```

Después:

```

10 public EventoBaloncesto(String id, String nombre, LocalDateTime fechaInicio,
11                          Equipos equipos) {
12     super(id, nombre, fechaInicio);
13     this.equipos = equipos;
14 }

10 public EventoFutbol(String id, String nombre, LocalDateTime fechaInicio,
11                      Equipos equipos) {
12     super(id, nombre, fechaInicio);
13     this.equipos = equipos;
14 }
15

10 public EventoTenis(String id, String nombre, LocalDateTime fechaInicio,
11                     Equipos equipos) {
12     super(id, nombre, fechaInicio);
13     this.equipos = equipos;
14 }
15

```

3. Justificación de cambios: Este cambio se realiza ya que anteriormente los parámetros del constructor de las clases, EventoFutbol, EventoBaloncesto y EventoTenis, estaban con mas de 4 parametros, lo que causaba el smell Long Parameter List, así que se aplicó la técnica Introduce Parameter Object. Los beneficios fueron: mejorar la legibilidad del código, a la hora de llamar al constructor además de que al convertirlo en objeto, se reduce el uso de parámetros primitivos en el constructor.

4. Verificación mediante pruebas: Se realizó la verificación mediante las pruebas CP-01, CP-02, CP-03, CP-04, CP-05, CP-06, CP-07, CP-08, CP-09, CP-10, CP-11, CP-14 y CP-15, ya que en todas estas pruebas se construye el evento con new Equipos("EquipoLocal", "EquipoVisitante"), al realizar las pruebas se verifica que este cambio no afecta al resultado original y sigue retornando los mismos resultados, por lo tanto esta refactorización está comprobada.

Move Field

1. Code smell corregido: Shotgun Surgery

2. Fragmento de código antes y después de la refactorización

Antes

```
...va  AdministradorTest.java x  CreadorPronosticoFutbolTest.java x  pronosticoFutbol.java [-/M] x
Source  History  [Icons]

1  package com.sportspredictor.factory;
2
3  import com.sportspredictor.shared.EstadoPronostico;
4  import com.sportspredictor.shared.EventoFutbol;
5  import com.sportspredictor.shared.Pronostico;
6  import com.sportspredictor.shared.Usuario;
7
8  public class pronosticoFutbol implements Pronostico {
9      private final EventoFutbol evento;
10     private final Usuario usuario;
11     private final String prediccionGanador;
12     private int marcadorLocal;
13     private int marcadorVisitante;
14     private EstadoPronostico estado;
15
16     public pronosticoFutbol(EventoFutbol evento, Usuario usuario, String prediccionGanador) {
17         this.evento = evento;
18         this.usuario = usuario;
19         this.prediccionGanador = prediccionGanador;
20         this.marcadorLocal = 0;
21         this.marcadorVisitante = 0;
22         this.estado = EstadoPronostico.PENDIENTE;
23     }
24 }
```

```
...M]  creadorPronosticoFutbol.java [-/M] x  creadorPronostico.java [-/M] x  pronosticoBaloncesto.java [-/M] x
Source  History  [Icons]

1  package com.sportspredictor.factory;
2
3  import com.sportspredictor.shared.EstadoPronostico;
4  import com.sportspredictor.shared.EventoBaloncesto;
5  import com.sportspredictor.shared.Pronostico;
6  import com.sportspredictor.shared.Usuario;
7
8  public class pronosticoBaloncesto implements Pronostico {
9      private final EventoBaloncesto evento;
10     private final Usuario usuario;
11     private final String prediccionGanador;
12     private int puntuacionTotal;
13     private EstadoPronostico estado;
14
15     public pronosticoBaloncesto(EventoBaloncesto evento, Usuario usuario, String prediccionGana
16         this.evento = evento;
17         this.usuario = usuario;
18         this.prediccionGanador = prediccionGanador;
19         this.puntuacionTotal = 0;
20         this.estado = EstadoPronostico.PENDIENTE;
21     }
22 }
```

```
...M]  creadorPronostico.java [-/M] x  pronosticoBaloncesto.java [-/M] x  pronosticoTenis.java [-/M] x
Source  History  [Icons]

8  public class pronosticoTenis implements Pronostico {
9      private final EventoTenis evento;
10     private final Usuario usuario;
11     private final String ganadorPronosticado;
12     private final int setsPronosticados;
13     private int setsResultado;
14     private EstadoPronostico estado;
15
16     public pronosticoTenis(EventoTenis evento, Usuario usuario, String datos) {
17         this.evento = evento;
18         this.usuario = usuario;
19
20         if (datos != null && datos.contains(",")) {
21             String[] partes = datos.split(",");
22             this.ganadorPronosticado = partes[0].trim();
23             this.setsPronosticados = partes.length > 1 ? parseSets(partes[1].trim()) : 0;
24         } else {
25             this.ganadorPronosticado = datos != null ? datos.trim() : "";
26             this.setsPronosticados = 0;
27         }
28
29         this.setsResultado = 0;
30         this.estado = EstadoPronostico.PENDIENTE;
31     }
32 }
```

Después:

```
...M] pronosticoBaloncesto.java [-/M] x pronosticoTenis.java [-/M] x AbstractPronostico.java x
Source History
14  */
15  public abstract class AbstractPronostico implements Pronostico{
16      protected final Usuario usuario;
17      protected EstadoPronostico estado;
18
19      protected AbstractPronostico(Usuario usuario){
20          this.usuario = usuario;
21          this.estado = EstadoPronostico.PENDIENTE;
22      }
}
```

```
...M] pronosticoTenis.java [-/M] x AbstractPronostico.java x pronosticoBaloncesto.java x
Source History
8  public class pronosticoBaloncesto extends AbstractPronostico {
9      private final EventoBaloncesto evento;
10     private final String prediccionGanador;
11     private int puntuacionTotal;
12
13     public pronosticoBaloncesto(EventoBaloncesto evento, Usuario usuario, String prediccionGanador) {
14         super(usuario);
15         this.evento = evento;
16         this.prediccionGanador = prediccionGanador;
17         this.puntuacionTotal = 0;
18     }
}
```

```
...M] pronosticoTenis.java [-/M] x AbstractPronostico.java x pronosticoBaloncesto.java x pronosticoFutbol.java x
Source History
9  public class pronosticoFutbol extends AbstractPronostico{
10     private final EventoFutbol evento;
11     private final String prediccionGanador;
12     private int marcadorLocal;
13     private int marcadorVisitante;
14
15     public pronosticoFutbol(EventoFutbol evento, Usuario usuario, String prediccionGanador) {
16         super(usuario);
17         this.evento = evento;
18         this.prediccionGanador = prediccionGanador;
19         this.marcadorLocal = 0;
20         this.marcadorVisitante = 0;
21     }
}
```

```
...M] AbstractPronostico.java x pronosticoBaloncesto.java x pronosticoFutbol.java x pronosticoTenis.java x
Source History
9  public class pronosticoTenis extends AbstractPronostico {
10     private final EventoTenis evento;
11     private final String ganadorPronosticado;
12     private final int setsPronosticados;
13     private int setsResultado;
14
15     public pronosticoTenis(EventoTenis evento, Usuario usuario, String datos) {
16         super(usuario);
17         this.evento = evento;
18         ResultadoTenis prediccion = ResultadoTenis.parse(datos);
19         this.ganadorPronosticado = prediccion.getGanador();
20         this.setsPronosticados = prediccion.getSets();
21         this.setsResultado = 0;
22     }
}
```

3. Justificación de cambios: Se realizan estos cambios ya que se detectó un shotgun surgery como smell en las tres clases, pronosticoFutbol, pronosticoBaloncesto y pronosticoTenis, principalmente en sus datos como Usuario y Estado, ya que estos datos se repetían en las tres clases, al igual que la inicialización del estadoPronostico.PENDIENTE, en dado caso que se realicen cambios estos obligarían a realizar pequeñas modificaciones en las 3 clases, por esta razón se aplicó Move Field el cual trasladó los datos como Usuario y Estado a una superclase llamada AbstractPronostico, concentrando los cambios en un solo lugar.

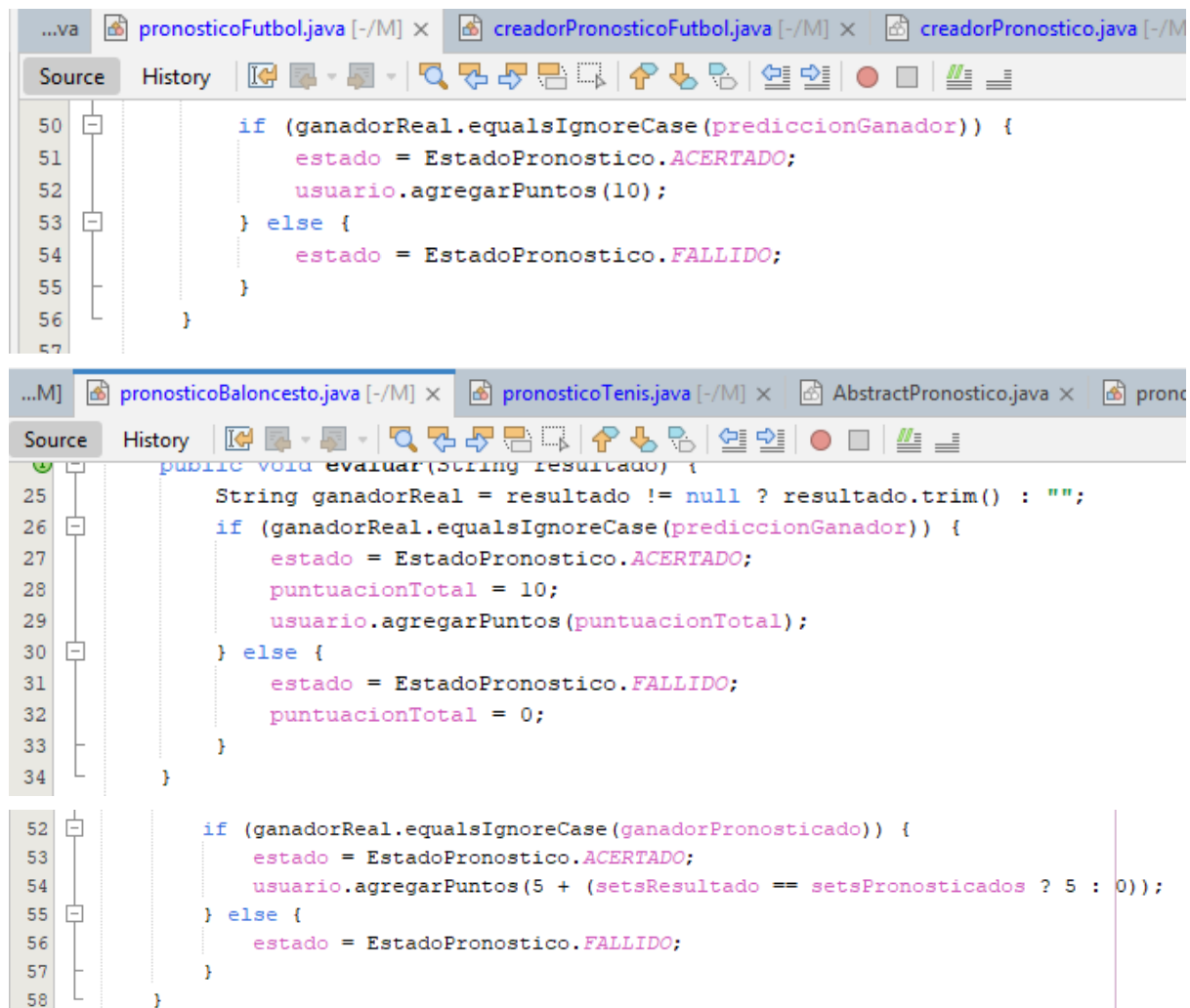
4. Verificación mediante pruebas: En las pruebas CP-01, CP-04, CP-11, CP-22 y CP-23, se está llamando al método obtenerEstado(), el valor obtenido no cambia, al realizar las pruebas se verifica que los resultados de estado siguen siendo los mismo, por lo tanto esta refactorización está comprobada.

Move Method

1. Code smell corregido: Shotgun Surgery

2. Fragmento código antes y después de la refactorización

Antes:



```
50         if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
51             estado = EstadoPronostico.ACERTADO;
52             usuario.agregarPuntos(10);
53         } else {
54             estado = EstadoPronostico.FALLIDO;
55         }
56     }
57 }
```

```
25     String ganadorReal = resultado != null ? resultado.trim() : "";
26     if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
27         estado = EstadoPronostico.ACERTADO;
28         puntuacionTotal = 10;
29         usuario.agregarPuntos(puntuacionTotal);
30     } else {
31         estado = EstadoPronostico.FALLIDO;
32         puntuacionTotal = 0;
33     }
34 }
```

```
52     if (ganadorReal.equalsIgnoreCase(ganadorPronosticado)) {
53         estado = EstadoPronostico.ACERTADO;
54         usuario.agregarPuntos(5 + (setsResultado == setsPronosticados ? 5 : 0));
55     } else {
56         estado = EstadoPronostico.FALLIDO;
57     }
58 }
```

Después:

```
...M] | pronosticoBaloncesto.java [-/M] x | pronosticoTenis.java [-/M] x | AbstractPronostico.java x
Source | History | [Icons]
18
19 | protected AbstractPronostico(Usuario usuario){
20 |     this.usuario = usuario;
21 |     this.estado = EstadoPronostico.PENDIENTE;
22 | }
23
24 | protected void registrarAcierto(int puntos){
25 |     estado = EstadoPronostico.ACERTADO;
26 |     usuario.agregarPuntos(puntos);
27 | }
28
29 | protected void registrarFallo(){
30 |     estado = EstadoPronostico.FALLIDO;
31 | }
32
33 | protected void registrarEnRevision(){
34 |     estado = EstadoPronostico.EN_REVISION;
35 | }
36
37 | @Override
38 | public EstadoPronostico obtenerEstado(){
39 |     return estado;
40 | }
41
42 | }
43
44
45 | String ganadorReal = resultado != null ? resultado.trim() : "";
46 | if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
47 |     puntuacionTotal = 10;
48 |     registrarAcierto(puntuacionTotal);
49 | } else {
50 |     puntuacionTotal = 0;
51 |     registrarFallo();
52 | }
53 | }
54
55
56 | if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
57 |     registrarAcierto(10);
58 | } else {
59 |     registrarFallo();
60 | }
61 | }
62
63
64 | if (ganadorReal.equalsIgnoreCase(ganadorPronosticado)) {
65 |     registrarAcierto(5 + (setsResultado == setsPronosticados ? 5 : 0));
66 | } else {
67 |     registrarFallo();
68 | }
69 | }
70
```

3. Justificación de cambios: En las tres clases se podía observar que se repetía el mismo patrón de transformar el estado en acertado, fallido y en revisión, además del método obtenerEstado(), por esta razón al usar la técnica Move Method se transporta todo este comportamiento y métodos de estado a la super clase AbstractPronostico, así solo se tiene que llamar al método protegido, reduciendo los cambios a una sola clase.

4. Verificación mediante pruebas: En las pruebas CP-01, CP-04, CP-06, CP-07, CP-08, CP-09, CP-10, CP-11 y CP-23, se está llamado al método obtenerEstado() y en algunas se llama también al método evaluar(), al ejecutar las pruebas retornan el mismo resultado que antes de la refactorización, por lo tanto se comprobó que mover los métodos de clases no altera el funcionamiento de la prueba.

Replace Primitive with Object

1. Code smell corregido: Primitive Obsession

2. Fragmento del código antes y después:

Antes:

```
25      @Override
26      public void evaluar(String resultado) {
27          if (resultado == null || !resultado.contains("-")) {
28              estado = EstadoPronostico.EN_REVISION;
29              return;
30          }
31
32          String[] partes = resultado.trim().split("-");
33          try {
34              marcadorLocal = Integer.parseInt(partes[0].trim());
35              marcadorVisitante = Integer.parseInt(partes[1].trim());
36          } catch (NumberFormatException e) {
37              estado = EstadoPronostico.EN_REVISION;
38              return;
39          }
40
41          String ganadorReal;
42          if (marcadorLocal > marcadorVisitante) {
43              ganadorReal = evento.getEquipoLocal();
44          } else if (marcadorLocal < marcadorVisitante) {
45              ganadorReal = evento.getEquipoVisitante();
46          } else {
47              ganadorReal = "EMPATE";
48          }
49      }
```

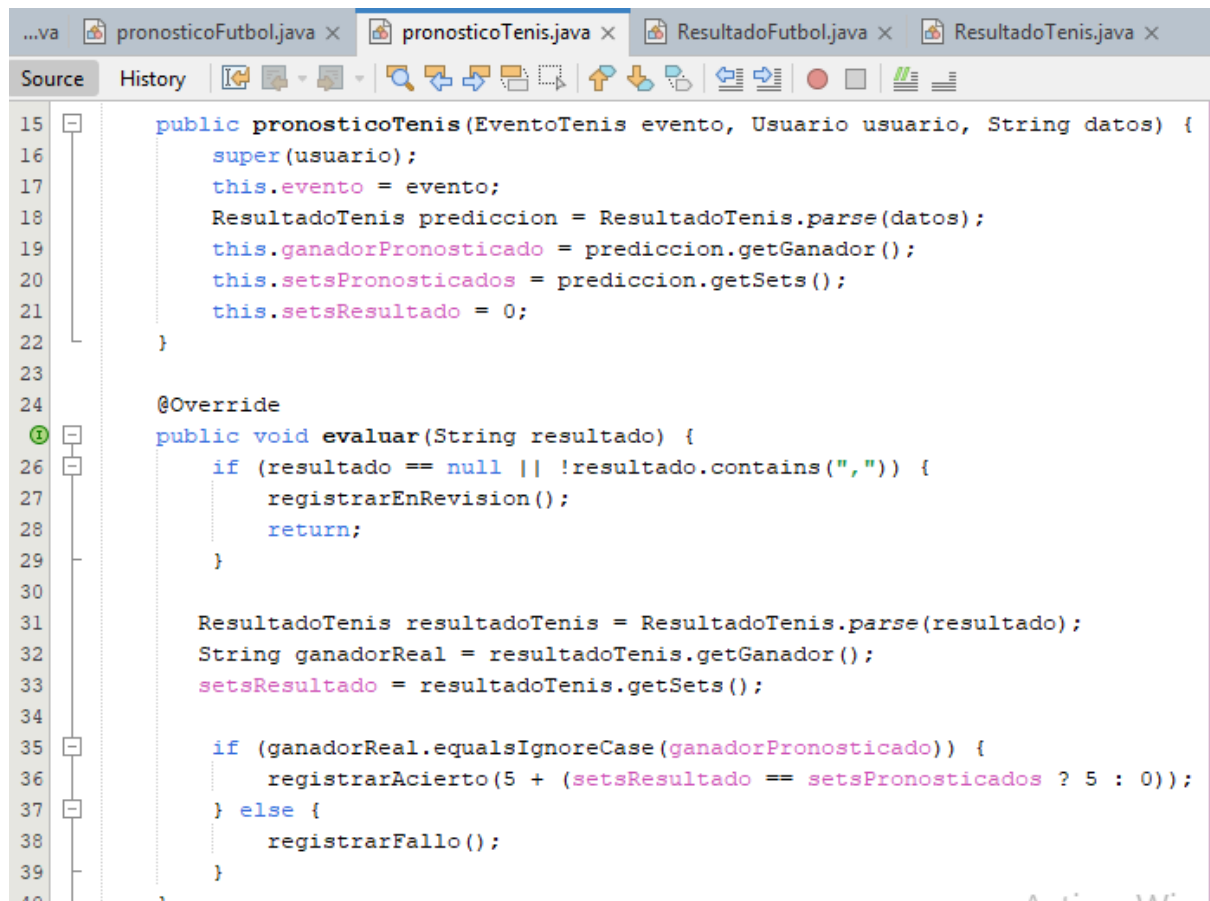
```
...M] pronosticoBaloncesto.java [-/M] x pronosticoTenis.java [-/M] x AbstractPronostico.java x pronosticoBalor
Source History
40
41 @Override
42 public void evaluar(String resultado) {
43     if (resultado == null || !resultado.contains(",")) {
44         estado = EstadoPronostico.EN_REVISION;
45         return;
46     }
47
48     String[] partes = resultado.trim().split(",");
49     String ganadorReal = partes[0].trim();
50     setsResultado = partes.length > 1 ? parseSets(partes[1].trim()) : 0;
51
52     if (ganadorReal.equalsIgnoreCase(ganadorPronosticado)) {
53         estado = EstadoPronostico.ACERTADO;
54         usuario.agregarPuntos(5 + (setsResultado == setsPronosticados ? 5 : 0));
55     } else {
56         estado = EstadoPronostico.FALLIDO;
57     }
58 }
```

Después:

```
...va pronosticoBaloncesto.java x pronosticoFutbol.java x pronosticoTenis.java x ResultadoFutbol.java
Source History
11 public class ResultadoFutbol {
12     private final int marcadorLocal;
13     private final int marcadorVisitante;
14
15     private ResultadoFutbol(int marcadorLocal, int marcadorVisitante){
16         this.marcadorLocal=marcadorLocal;
17         this.marcadorVisitante= marcadorVisitante;
18     }
19
20     public static ResultadoFutbol parse(String resultado){
21         if(resultado == null || !resultado.contains("-")){
22             return null;
23         }
24         String[] partes = resultado.trim().split("-");
25         try{
26             int local = Integer.parseInt(partes[0].trim());
27             int visitante = Integer.parseInt(partes[1].trim());
28             return new ResultadoFutbol(local, visitante);
29         } catch (NumberFormatException e){
30             return null;
31         }
32     }
33
34     public String determinarGanador(Equipos equipos){
35         if(marcadorLocal>marcadorVisitante){
36             return equipos.getNombreLocal();
37         }
38     }
39 }
```

```
...va  pronosticoFutbol.java x  pronosticoTenis.java x  ResultadoFutbol.java x  ResultadoTenis.java x
Source History
11 public class ResultadoTenis {
12     private final String ganador;
13     private final int sets;
14
15     private ResultadoTenis(String ganador, int sets){
16         this.ganador = ganador;
17         this.sets = sets;
18     }
19
20     public static ResultadoTenis parse(String datos){
21         if(datos!=null && datos.contains(",")){
22             String[] partes = datos.split(",");
23             String ganadorParseado = partes[0].trim();
24             int setsParseados = partes.length > 1 ? parseSets(partes[1].trim()): 0;
25             return new ResultadoTenis (ganadorParseado,setsParseados);
26         }
27         String ganadorParseado = datos != null ? datos.trim(): "";
28         return new ResultadoTenis(ganadorParseado, 0);
29     }
30     private static int parseSets(String texto){
31         try{
32             return Integer.parseInt(texto);
33         }catch (NumberFormatException e){
34             return 0;
35         }
36     }
37 }
```

```
...va  pronosticoFutbol.java x  pronosticoTenis.java x  ResultadoFutbol.java x  ResultadoTenis.java x
Source History
22
23 @Override
24 public void evaluar(String resultado) {
25     ResultadoFutbol resultadoFutbol = ResultadoFutbol.parse(resultado);
26     if (resultadoFutbol == null) {
27         registrarEnRevision();
28         return;
29     }
30     marcadorLocal = resultadoFutbol.getMarcadorLocal();
31     marcadorVisitante = resultadoFutbol.getMarcadorVisitante();
32
33     String ganadorReal = resultadoFutbol.determinarGanador(evento.getEquipos());
34
35     if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
36         registrarAcierto(10);
37     } else {
38         registrarFallo();
39     }
40 }
41 }
```



```
15 public pronosticoTenis(EventoTenis evento, Usuario usuario, String datos) {
16     super(usuario);
17     this.evento = evento;
18     ResultadoTenis prediccion = ResultadoTenis.parse(datos);
19     this.ganadorPronosticado = prediccion.getGanador();
20     this.setsPronosticados = prediccion.getSets();
21     this.setsResultado = 0;
22 }
23
24 @Override
25 public void evaluar(String resultado) {
26     if (resultado == null || !resultado.contains(",")) {
27         registrarEnRevision();
28         return;
29     }
30
31     ResultadoTenis resultadoTenis = ResultadoTenis.parse(resultado);
32     String ganadorReal = resultadoTenis.getGanador();
33     setsResultado = resultadoTenis.getSets();
34
35     if (ganadorReal.equalsIgnoreCase(ganadorPronosticado)) {
36         registrarAcierto(5 + (setsResultado == setsPronosticados ? 5 : 0));
37     } else {
38         registrarFallo();
39     }
40 }
```

3. Justificación de cambios: Se realizan estos cambios ya que el dato resultado era un dato primitivo, un string, si había un error al entregar el formato de resultado, entonces el programa dará resultados incorrectos sobre el pronóstico, por esta razón se utilizó la técnica Replace Primitive with Object, convirtiendo Resultado en un objeto complejo, moviendo toda la lógica que estaba dentro del método evaluar al nuevo objeto, de esta forma se ganan beneficios como, si es que se modifica el formato en un futuro, entonces solo se necesitara modificar la clase Resultado, además de reducir el tamaño del método evaluar para mejorar la legibilidad de este método.

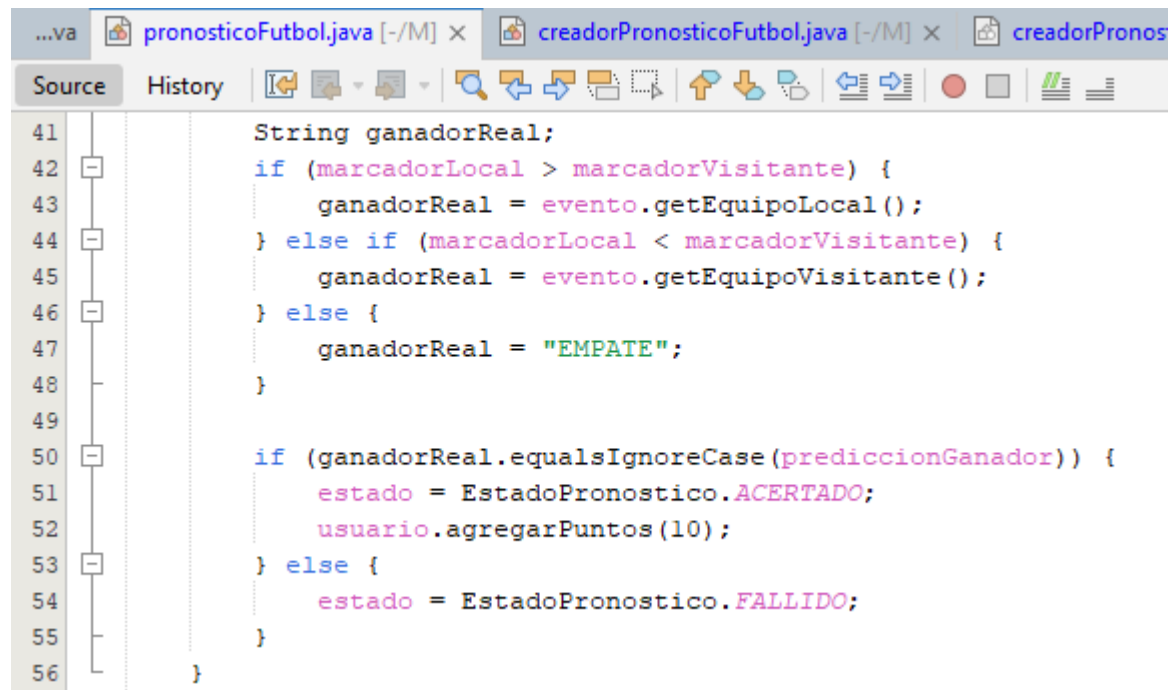
4. Verificación mediante pruebas: En las pruebas CP-04, CP-05 y CP-06, hace uso del dato Resultado para realizar sus pruebas, si se ejecutan estas pruebas después de la refactorización, se puede observar que siguen devolviendo el mismo resultado, adecuado a sus casos de prueba, por lo tanto se confirma que la refactorización está correcta.

Extract Method y Move Method

1. Code smell corregido: Feature Envy

2. Fragmento de código antes y después de la refactorización

Antes:



The screenshot shows an IDE with three tabs: `...va`, `pronosticoFutbol.java [-/M] x`, and `creadorPronosticoFutbol.java [-/M] x`. The `pronosticoFutbol.java` tab is active, displaying the following Java code:

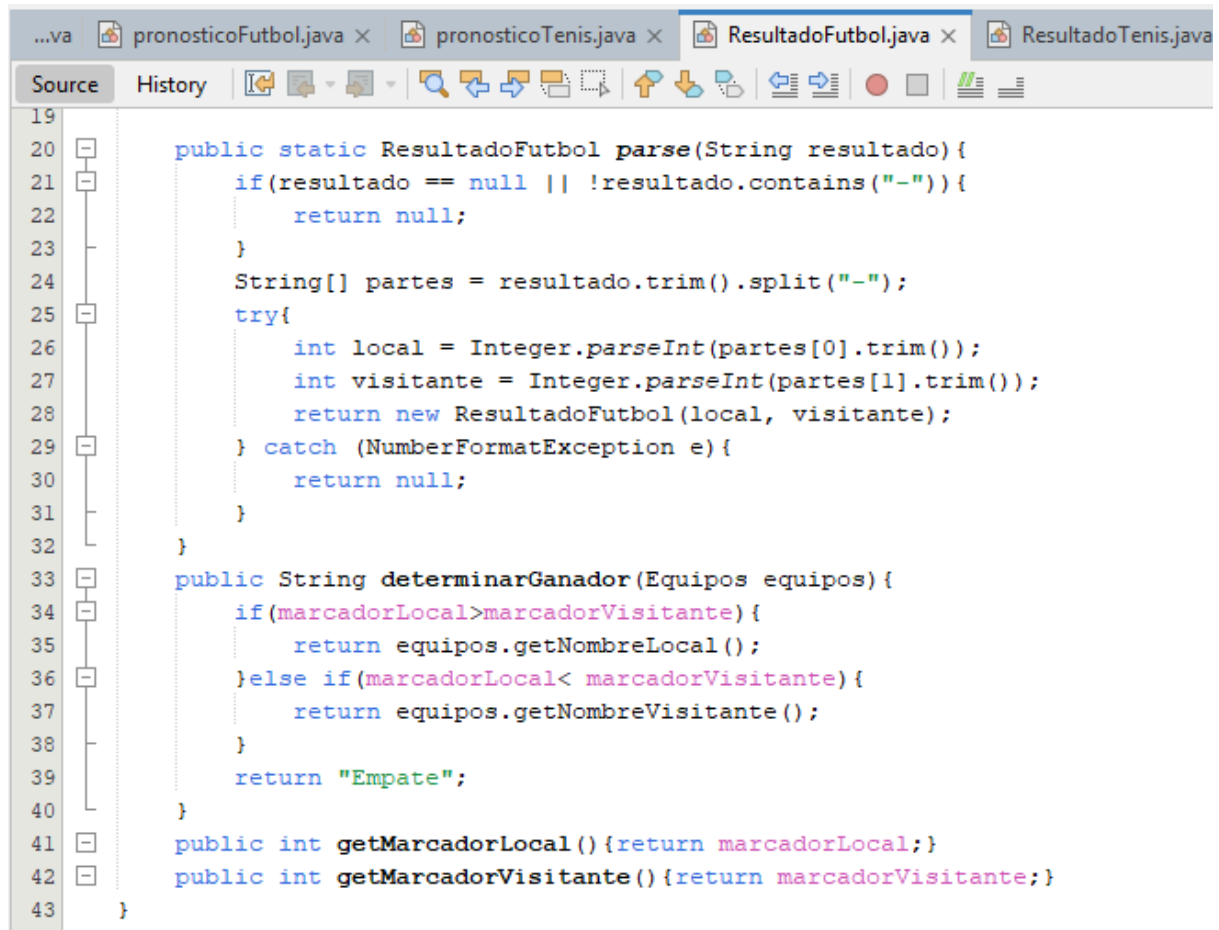
```
41 String ganadorReal;
42 if (marcadorLocal > marcadorVisitante) {
43     ganadorReal = evento.getEquipoLocal();
44 } else if (marcadorLocal < marcadorVisitante) {
45     ganadorReal = evento.getEquipoVisitante();
46 } else {
47     ganadorReal = "EMPATE";
48 }
49
50 if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
51     estado = EstadoPronostico.ACERTADO;
52     usuario.agregarPuntos(10);
53 } else {
54     estado = EstadoPronostico.FALLIDO;
55 }
56 }
```

Después:

```

30     marcadorLocal = resultadoFutbol.getMarcadorLocal();
31     marcadorVisitante = resultadoFutbol.getMarcadorVisitante();
32
33     String ganadorReal = resultadoFutbol.determinarGanador(evento.getEquipos());
34
35     if (ganadorReal.equalsIgnoreCase(prediccionGanador)) {
36         registrarAcierto(10);
37     } else {
38         registrarFallo();
39     }
40 }

```



```

19
20 public static ResultadoFutbol parse(String resultado) {
21     if (resultado == null || !resultado.contains("-")) {
22         return null;
23     }
24     String[] partes = resultado.trim().split("-");
25     try {
26         int local = Integer.parseInt(partes[0].trim());
27         int visitante = Integer.parseInt(partes[1].trim());
28         return new ResultadoFutbol(local, visitante);
29     } catch (NumberFormatException e) {
30         return null;
31     }
32 }
33
34 public String determinarGanador(Equipos equipos) {
35     if (marcadorLocal > marcadorVisitante) {
36         return equipos.getNombreLocal();
37     } else if (marcadorLocal < marcadorVisitante) {
38         return equipos.getNombreVisitante();
39     }
40     return "Empate";
41 }
42
43 public int getMarcadorLocal() { return marcadorLocal; }
44 public int getMarcadorVisitante() { return marcadorVisitante; }
45 }

```

3. Justificación de cambios: Se realizan estos cambios ya que en el método evaluar() se estaba accediendo más a los datos de la clase Evento que a los propios metodos de pronosticoFutbol, por lo tanto se aplica una combinación de dos técnicas, Extract Method y Move Method, una extrae el fragmento del código dentro del método y la otra lo mueve a su nueva clase, en este caso es ResultadoFutbol. Los beneficios de realizar esto son que se reduce la responsabilidad del método evaluar() ya que toda la lógica de decisiones queda guardada en ResultadoFutbol, mejorando la legibilidad del código dentro de evaluar().

4. Verificación mediante pruebas: En las pruebas CP-04, CP-05, CP-06 y CP-23 hacen uso de los métodos de resultadoFutbol, como determinarGanador(), además de hacer uso de evaluar(), al ejecutar estas pruebas, se puede verificar que el resultado que retornan es el mismo, por lo tanto se confirma que la refactorización está correcta.

- 1.Code smell corregido:**Alternative Classes with Different Interfaces
- 2. Técnica de refactorización:** Unificar Interfaces (Unify Interfaces)
- 3.Justificación de cambios:** Se hara esta refactorización ya que

Enlace del repositorio

<https://github.com/nezouein/Tarea03-Refactoring-Pruebas/tree/main>